Project 06: Job Runner

Concept notes for my book. The core ideas and the ones that confused me.

How to use these notes

- Copy the **Key idea** boxes and the diagrams by hand. Copying a diagram forces me to understand every arrow.
- Do not copy the self-test answers, because there are none. Answer them from memory the day *after* copying. If I have to look something up, that is a gap: write it in gaps.md.
- Read each concept as a **timeline**, not as code. Concurrent code does not run top to bottom.
- There is no code in these notes on purpose. The code belongs in my head, not on paper.

Contents

1. The database is the referee
2. The job lifecycle: state transitions as mutual exclusion
3. Claiming: row locks and SKIP LOCKED
4. Leases, heartbeats and stale reclaim
5. Fencing tokens
6. At-least-once execution and idempotency
7. Retries and exponential backoff
8. Indexes, ORDER BY and EXPLAIN
9. Cancellation inside the worker
10. Scope lifetime and the using declaration
11. Reading concurrent code: the four timelines
12. Current state vs history

Appendix A: side lessons from today (IDOR, observability)

Appendix B: words to search

Appendix C: the final exam

Appendix D: primary docs to read

1. The database is the referee

> **KEY IDEA**
> Two processes share no memory. Any rule that must hold **across** workers can only be enforced by the one thing they all share: the database. A C# *lock* or an *if* only protects the process it runs in.

```
                                +----------------+
client --POST--> API --INSERT-->|                |<-- claim / beat / done --  Worker 1
                                |   jobs table   |<-- claim / beat / done --  Worker 2
                                |  (PostgreSQL)  |<-- claim / beat / done --  Worker 3
                                +----------------+

The API and the workers never talk to each other.
The jobs table is the ONLY shared state.
```

The broken way: check in C#, then act

Worker A reads job 7 and sees *Pending*. Worker B reads job 7 a millisecond later and also sees *Pending*. Both decide "it's free, I'll take it". Both run it. The check was true when each worker read it, but stopped being true before they acted.

```
t0   A: read job 7 -> Pending
t1   B: read job 7 -> Pending
t2   A: set Running, run job 7
t3   B: set Running, run job 7        <-- duplicate execution
```

This is a **read-modify-write race**, also called **check-then-act** or **TOCTOU** (time of check to time of use). The fix is always the same shape: make the check and the change **one atomic step inside the database**, so there is no gap between them.

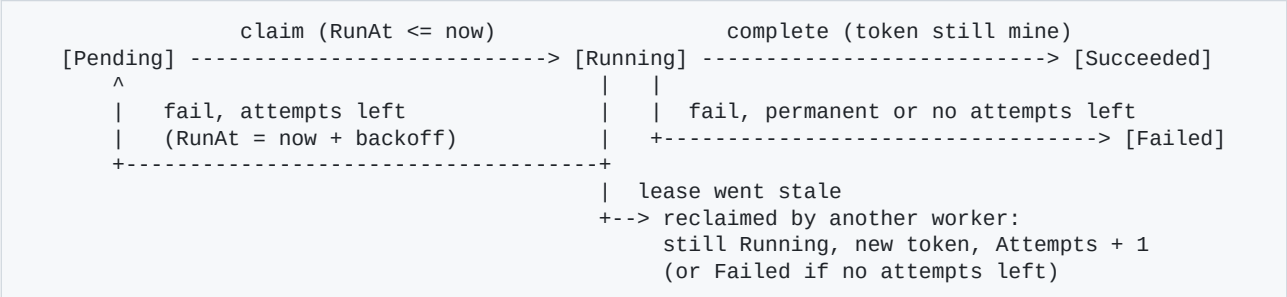
> **TRAP**
> "I check the status before updating, so it's safe." No. Anything that reads, then decides in C#, then writes, has a gap between the read and the write. Another process can act inside that gap.

Self-test (answer from memory, the next day)

1. Why can't a C# lock stop two workers from running the same job?
2. Draw the check-then-act timeline for two workers from memory.
3. What does TOCTOU stand for, and what is the general shape of the fix?

2. The job lifecycle: state transitions as mutual exclusion

> **KEY IDEA**
> A worker owns a job because it moved the row from one state to another in a **single conditional UPDATE**: "change it only if it is still in the state I expect". Only one UPDATE can win that transition. If my UPDATE changed **0 rows**, someone else got there first.



Every arrow in this diagram is one UPDATE whose WHERE clause names the state it expects to leave. That WHERE clause is the lock. There is no separate locking step.

Reading the result of an UPDATE

Rows changed	Meaning	What the worker does
1	My transition won.	Carry on: I own the job, or my result was recorded.
0	The row was not in the state I expected. Someone else changed it first.	Do NOT retry blindly. Log it and walk away: the job belongs to someone else now.

> **TRAP**
> Ignoring the row count. An UPDATE that matched nothing does not throw an exception. If I don't check the count, I silently believe I did something I didn't.

Self-test (answer from memory, the next day)

1. Draw the lifecycle diagram with every arrow labelled.
2. Why is "0 rows changed" not an error, and what does it tell me?
3. Which column decides whether a job can be claimed yet?

3. Claiming: row locks and SKIP LOCKED

> **KEY IDEA**
> **FOR UPDATE** locks the rows a SELECT picks, so no other transaction can lock or change them until mine ends. **SKIP LOCKED** tells other workers to *pass over* locked rows instead of waiting for them. Together: each worker grabs a different batch, nobody waits.

Two workers, same millisecond

```
Pending jobs: 1 2 3 4 5 6 7 8 9 10 11 12 ... (ordered by RunAt, Id)

t0      A: pick first 10 pending, FOR UPDATE      -> locks 1..10
t0+1ms  B: pick first 10 pending, FOR UPDATE ...

...without SKIP LOCKED:
        B: wants row 1 -> it is locked -> B WAITS
        A: marks 1..10 Running, commits, locks released
        B: wakes up, re-checks row 1 -> no longer Pending -> skips it
        Result: B wasted its time queueing behind A. With 3 workers,
                they take turns instead of working in parallel.

...with NOWAIT:
        B: row 1 is locked -> B gets an ERROR immediately.

...with SKIP LOCKED:
        B: rows 1..10 locked -> pass over them -> locks 11..20
        Result: A has 1..10, B has 11..20, nobody waited, nobody overlapped.
```

Why the SELECT and the UPDATE are one statement

A row lock only lasts until the end of the transaction. If I ran "select and lock" as one statement and "mark them Running" as a second statement, each in its own auto-committed transaction, the locks would be released **between** the two. Another worker could slip into that gap. Putting the SELECT inside the UPDATE makes "find, lock, mark Running, return" a single atomic step.

SKIP LOCKED is not a general tool

It deliberately gives an **inconsistent view** of the table: "the first 10 pending rows *that nobody else is holding*". That is perfect for a queue, where any free job will do. It is wrong for a report or a balance, where I need every row.

In my code: JobStore.ClaimAsync (new jobs) and JobStore.ReclaimStaleAsync (stale jobs). Both use the same pattern.

Self-test (answer from memory, the next day)

1. Two workers run the claim query in the same millisecond. Why does neither get the other's rows?
2. What would happen without SKIP LOCKED? With NOWAIT?
3. Why must "find and lock" and "mark Running" happen in one statement?
4. Name one situation where SKIP LOCKED would give a wrong answer.

4. Leases, heartbeats and stale reclaim

> **KEY IDEA**
> A database lock ends with the transaction, and a job runs for much longer than that. So ownership of a running job is a **lease**: a timestamp in the row that the worker keeps renewing. If it is not renewed within the **stale timeout**, any worker may take the job.

	Row lock (FOR UPDATE)	Lease (ClaimedAt + timeout)
Held by	The database, for a transaction	A worker, written into the row
Lasts	Milliseconds: until commit	As long as the worker keeps renewing it
Ends when	Commit, rollback or disconnect	Time runs out without a renewal
Survives a crash?	No: released automatically	Yes: stays until it expires
Used for	Claiming: who grabs which rows	Running: who is working on the job

The timeline of a crash

```
t=0s    A claims job 5. ClaimedAt = 0s.
t=10s   A heartbeats.   ClaimedAt = 10s.
t=12s   A is killed (kill -9). No more heartbeats.
t=40s   ClaimedAt (10s) is now older than 30s -> job 5 is STALE.
t=45s   B's reclaim check runs (every 15s): takes job 5, new token, Attempts + 1.
t=50s   B finishes job 5.
```

The timing rule

My worker uses: **beat deadline 3s < heartbeat every 10s <= stale timeout 30s / 3**.

- **Heartbeat <= stale / 3**: a worker gets about three chances to renew before it can be robbed. Missing one or two beats (a slow network, a GC pause) does not lose the job.
- **Beat deadline < heartbeat interval**: a heartbeat call that hangs is abandoned before the next one is due. Without this, one stuck call could silently eat the whole safety margin.
- **Two failed heartbeats in a row**: the worker stops itself. It can no longer prove it owns anything, so it is safer to stop than to keep working on jobs that may already be someone else's.

> **TRAP**
> A lease expiring does **not** stop the old worker. It only lets someone else in. The old worker may be alive and still running the job. That is exactly why fencing (next section) exists.

In my code: JobStore.HeartbeatAsync, JobStore.ReclaimStaleAsync, and the timing constants at the top of JobWorker.

Self-test (answer from memory, the next day)

1. What is the difference between a lock and a lease? Give three differences.
2. Draw the crash timeline. At what moment does job 5 become stale, and when is it reclaimed?
3. Why must the heartbeat interval be at most a third of the stale timeout?
4. Why does the worker stop itself after two failed heartbeats?

5. Fencing tokens

> **KEY IDEA**
> Every claim **increments a number** on the row (the fence token) and gives it to the worker. Every later write by that worker says "only if the token is still mine". A worker that has been replaced holds an old number, so its writes match **0 rows** and do nothing.

The hung worker (my answer to "You've got it when" #2)

Step	Status	FenceToken	A holds	B holds
A claims job 5	Running	1	1	-
A freezes (GC pause, SIGSTOP). Heartbeats stop.	Running	1	1	-
Lease goes stale. B reclaims.	Running	2	1	2
B completes: only if token = 2. Matches.	Succeeded	2	1	2
A wakes, completes: only if token = 1. Matches 0 rows.	Succeeded	2	1	2

A's late write is a no-op. The row keeps B's result. A logs "lost fence token ownership" and moves on.

Why not just check LockedBy (the worker's name)?

A name can repeat; a counter never does. Example from my own code: worker A's heartbeat is slow, A's job goes stale, and **A's own reclaim loop** picks the job up again. Now there are two runs inside worker A, the old one and the new one, and both have LockedBy = A. Only the token can tell them apart: old run holds 1, new run holds 2.

Where the token must be checked

On **every** write a worker makes after claiming: complete, fail and heartbeat. One unfenced write is enough to let a zombie worker overwrite a newer result.

The deeper idea (Martin Kleppmann)

A lock or lease alone cannot protect you, because the holder cannot know it has lost the lease while it is paused. The protection has to come from the **thing being written to** rejecting old tokens. In my system that thing is the jobs table, and only the jobs table.

In my code: The WHERE clauses of JobStore.CompleteAsync, FailAsync and HeartbeatAsync.

Self-test (answer from memory, the next day)

1. Fill in the hung-worker table from memory.
2. Name the column and the condition that stops worker A.
3. Give a scenario where LockedBy cannot tell two runs apart but the token can.
4. Why can't the worker itself detect that it was paused and lost the lease?

6. At-least-once execution and idempotency

KEY IDEA
The fence protects the **jobs row**, not the outside world. By the time A's write is rejected, A's handler has already run. So every job can run **more than once**, and every handler must be safe to run twice. That property is called **idempotency**.

```
A: claim (token 1) -> send email ..... freeze ............ wake -> complete(1): 0 rows
B:                              reclaim (token 2) -> send email -> complete(2): ok

jobs row:       correct (Succeeded, written once, by B)
customer inbox: TWO emails                          <-- the fence never saw this
```

Who checks the token, and who doesn't

Checks the token	Never sees the token
The jobs table: complete, fail and heartbeat writes all compare it.	Email provider, payment gateway, partner webhooks, the file system or bucket where a resized image is written. They cannot reject a second execution.

Two ways to protect a system that can't see my token

- **Idempotency key**: send the provider a value that is the same on every attempt. The provider remembers it and performs the operation only once. (Stripe's Idempotency-Key header is the classic example.)
- **Conditional write / compare-and-set**: storage that only accepts a write if the version I read is still current (search "ETag If-Match"). A stale writer is rejected, which is fencing applied to storage.

Exactly-once does not exist over a network

If a request times out, I cannot know whether it happened. So I either risk running it twice (at-least-once) or risk never running it (at-most-once). The practical answer: **at-least-once delivery + idempotent handlers = effectively once**.

TRAP
Thinking "my fence works, so jobs run once". Jobs are **recorded** once. They may be **executed** more than once.

Self-test (answer from memory, the next day)

1. Finish the sentence: "A handler is idempotent if ..."
2. For a welcome-email handler, what would it check, and where would it keep that information, so the second run sends nothing?
3. As an idempotency key, would I send the job ID or the fence token? Why? (The answers differ.)
4. Why is exactly-once delivery impossible over a network?

7. Retries and exponential backoff

> **KEY IDEA**
> Retry state lives **in the row**, not in memory, so it survives crashes. A failed job goes back to Pending with RunAt pushed into the future. The delay **doubles** each attempt so a struggling dependency is not hammered.

```
attempt:   1     2     3     4     5
wait:      1s    2s    4s    8s    16s     (2 to the power of attempts-1)
after attempt 5 fails -> Failed (MaxAttempts reached)
```

One column, two jobs

RunAt means "not before this time". It serves both **scheduled jobs** (run tomorrow at 9) and **backoff** (retry in 4 seconds). The claim query only needs one rule: RunAt has passed.

Retryable vs permanent failure

Kind	Example	What happens
Retryable	Timeout, network error, provider returns 503	Back to Pending with backoff
Permanent	No handler for this job type, payload can't be parsed	Straight to Failed. Retrying can never fix it, so retrying only wastes attempts.

Concepts I don't have yet

- **Jitter**: add randomness to the delay. Without it, 1000 jobs that failed together all retry at the same instant and fail together again (the **thundering herd**).
- **A cap**: a maximum delay, so attempt 20 doesn't wait 6 days.

Self-test (answer from memory, the next day)

1. Why does retry state have to live in the database and not in a C# variable?
2. Why exponential and not a fixed 5-second delay?
3. What is jitter, and what problem does it solve?
4. Give one retryable and one permanent failure, and say why they must be treated differently.

8. Indexes, ORDER BY and EXPLAIN

> **KEY IDEA**
> A B-tree index is a **list kept in sorted order**. If my ORDER BY matches that order, Postgres reads the first N entries and **stops**. If it doesn't match, Postgres must find **every** matching row and sort them all before it can return the first one (or fall back to a worse plan).

```
Index on (RunAt, Id), only rows where Status = Pending:

  (09:00, 4) (09:00, 9) (09:01, 2) (09:05, 7) (09:05, 8) ... 1,000,000 entries
  |<------ LIMIT 10 ------>|  stop here

ORDER BY RunAt, Id   -> walk the list from the start, take 10, stop.   ~10 rows read
ORDER BY Id, RunAt   -> this list is in the wrong order. Postgres must either
                        find all 1,000,000 matches and Sort them, or walk a
                        different index and filter row by row. EXPLAIN shows
                        which. Either way: far more than 10 rows read.
```

Column order matters

An index on (RunAt, Id) is sorted by RunAt first, and by Id only to break ties. It is a phone book sorted by surname then first name. Useless for "everyone called John, in order".

Partial indexes

My claim index only contains **Pending** rows. Millions of finished jobs are not in it at all, so it stays small and fast no matter how old the system gets. The stale-reclaim index does the same for **Running** rows, sorted by ClaimedAt.

Proving it with EXPLAIN

Plan node	Meaning
Index Scan using IX_jobs_claim	Good: it walks my index in order.
Limit above an Index Scan	Good: it stops after N rows.
Sort	Bad here: the index order didn't satisfy ORDER BY, so it sorts everything first.
Seq Scan	Bad here: it reads the whole table.

My experiment to run: EXPLAIN the claim query once with the correct ORDER BY and once with the columns swapped. Compare the two plans and look for the Sort node.

In my code: JobConfiguration (IX_jobs_claim, IX_jobs_stale) and tests/explain_test.sh.

Self-test (answer from memory, the next day)

1. Why does the claim query's ORDER BY have to match the index?
2. What would EXPLAIN show if it didn't?
3. With 1,000,000 pending jobs and LIMIT 10, roughly how many rows are read in each case?
4. What is a partial index, and why is it a good fit for a job queue?

9. Cancellation inside the worker

> **KEY IDEA**
> A running handler can be stopped for **three different reasons**. Each one needs a different reaction. The worker combines all three into one token for the handler, then uses **exception filters** (catch ... when ...) to work out which one fired.

Reason	Triggered by	Reaction	Why
Shutdown	The host is stopping	Leave the job Running	Stale reclaim will pick it up later. Writing anything now could race with shutdown.
Lease lost	Heartbeat saw the job missing	Do nothing to the row	Another worker owns it now. Any write would be fenced out anyway.
Timeout	Job ran longer than 60s	Mark failed (retryable)	I still own it, so it is my job to record the failure.

Linked tokens

A **linked** cancellation source cancels when **any** of its parents cancels. The handler only watches one token. The worker keeps the three parents so it can ask afterwards which one fired.

Exception filters

A catch with a **when** condition only catches the exception if the condition is true. Otherwise the exception keeps travelling up. That lets one exception type (OperationCanceledException) be handled three different ways depending on *why* it happened.

Today's bug: graceful shutdown

```
main loop:   claim -> nothing found -> sleep 1s (outside any try) ...
SIGTERM  ->  the sleep is cancelled -> exception
             -> escapes the whole loop -> "stopped gracefully" never logged

Before the fix: 3 of 8 shutdowns clean.  After: 12 of 12.
The host swallows that exception silently, so the logs show NO error. Only the missing
final log line gave it away.
```

> **TRAP**
> Catching every OperationCanceledException. That hides real bugs: a cancellation I didn't expect gets treated like one I did. Always filter on the token I expect.

Self-test (answer from memory, the next day)

1. Name the three cancellation reasons and the reaction to each. Why is each reaction different?
2. What does a linked cancellation source do?
3. What does "when" on a catch do? What happens to the exception if the condition is false?
4. What happens when Task.Delay is given a token that is already cancelled?

10. Scope lifetime and the using declaration

KEY IDEA
A DI **scope** owns the scoped services it creates, such as the DbContext. Disposing the scope disposes them. A **using declaration** disposes at the end of the enclosing block, which means after the catch and finally blocks, and also on an early return.

```
method starts
  create scope  ----------------------------+  (using declaration)
  get JobStore, handlers                    |
  early return?  -------------------------->|  disposed here
  try   { run handler, complete }           |
  catch { fail }   <- still needs JobStore   |
  finally { clean up }                       |
method ends  ------------------------------>+  disposed here, AFTER catch/finally
```

Today's bug

The per-job scope was created but never disposed. Be precise about how bad that was:

- **Not** a connection leak: EF Core opens and closes the connection around each command.
- **Not** a big memory leak: nothing kept a reference, so the GC collected it eventually.
- **Was** a lifetime bug: Dispose never ran. Any scoped service holding a real resource (a file, a transaction, a pooled context) would never have been released.

TRAP
Disposing too early. If the scope ended right after the handler ran, the catch blocks would call FailAsync on a disposed DbContext and throw ObjectDisposedException.

Self-test (answer from memory, the next day)

1. What does disposing a scope dispose?
2. With a using declaration at the top of a method, exactly when is it disposed?
3. Why would disposing the scope straight after the handler be a bug?
4. Using statement (with braces) vs using declaration: what's the difference?

11. Reading concurrent code: the four timelines

> **KEY IDEA**
> JobWorker is **four programs in one file** that run at the same time. Reading it top to bottom fails because it doesn't run top to bottom. Read it **one timeline at a time**, then ask where the timelines touch.

```
MAIN LOOP      claim batch -> process batch -> sleep -> claim -> ...
               every 15s: reclaim stale jobs

HEARTBEAT      tick (10s) -> renew my leases -> cancel jobs I lost -> tick -> ...

HANDLER        run job 5 ............................... complete / fail
                  ^ can be cancelled by: shutdown, heartbeat, timeout

DATABASE       rows changing under all three, AND under the other workers
```

Where the timelines touch: the held-jobs dictionary

The worker keeps a dictionary of the jobs it currently holds: job ID to (my token, a cancel switch). It is the only in-memory state shared between timelines, which is why it is a ConcurrentDictionary.

Who	Timeline	Does what to it
ProcessBatchAsync (start)	Main loop	Adds every claimed job
HeartbeatLoop	Heartbeat	Reads all entries to renew them; removes and cancels lost ones
ProcessJobAsync	Handler	Reads its own entry; removes it when the job ends (and on the no-handler path)
ProcessBatchAsync (finally)	Main loop	Removes anything left over

Four different places remove entries. To understand any one of them I must know the others exist. That is what made this file so hard to hold in my head.

How to read a file like this

- Draw the timelines as columns on paper, before reading any details.
- Pick **one scenario** (success, throw, SIGSTOP, shutdown) and trace it through the columns.
- For every shared variable, list every line that touches it and which timeline that line is on.
- Use the debugger: one breakpoint per timeline, then watch the order they actually fire in.

Self-test (answer from memory, the next day)

1. Draw the four timelines from memory.
2. List every place that removes an entry from the held-jobs dictionary.
3. Why is a plain Dictionary not safe here?

12. Current state vs history

> **KEY IDEA**
> A row that keeps getting UPDATEd holds only **the present**. Every update erases the old value. Questions about the past ("why did job 9812 take 40 minutes?") need **history**: an append-only table where rows are inserted and never overwritten.

The overloaded column in my schema

ClaimedAt means two things at once: "when this attempt was claimed" and "when the lease was last renewed", because the heartbeat overwrites it. Stale reclaim needs the second meaning. Measuring queue wait time needs the first. One column cannot serve both.

```
10:00  job created           CreatedAt = 10:00
10:05  claimed               ClaimedAt = 10:05     (really waited 5 min)
10:06  heartbeat             ClaimedAt = 10:06
 ...
10:40  heartbeat             ClaimedAt = 10:40
       ClaimedAt - CreatedAt = 40 min              <-- WRONG answer
```

The design I'd use: a job_attempts table

- One row per attempt, identified by (job ID, fence token). The token is already unique per claim, so it is a **natural key**.
- Inserted **in the same transaction as the claim**, so a crash between the two can't leave a running job with no attempt row. Same idea as the transactional outbox in project 05.
- Closed by complete, fail, or reclaim ("timed out").
- Needs a **retention policy**, because it grows forever.

Not built in project 06: it's an observability feature, and no acceptance criterion needs it. It is listed in the README under "What I'd do next".

Self-test (answer from memory, the next day)

1. Why can't an UPDATEd row answer questions about the past?
2. What two meanings does ClaimedAt carry, and which part of the system needs each one?
3. Why must the attempt row be inserted in the same transaction as the claim?
4. For attempt 3 of a job, which timestamps would I subtract to get its wait time?

Appendix A: side lessons from today

IDOR: why 404, not 403

> **KEY IDEA**
> When a user asks for a record they don't own, return **404**. A 403 says "this exists but you can't have it". An attacker can then try IDs 1, 2, 3 ... and map every record in the system.

- The owner check must use the user ID from the **authenticated identity** (the token's claims), never one the client sends in the route, query or body.
- 400 means the request itself is malformed. GET /expenses/42 is perfectly well formed, so 400 is wrong.
- Names: **IDOR** (Insecure Direct Object Reference), **BOLA** (Broken Object Level Authorization), number 1 in the OWASP API Security Top 10.

Observability: logging errors is not enough

Logging the exception tells me *that* a job failed, not *why it took 40 minutes*. To answer questions I didn't plan for, I need:

- A timestamp for every **state transition**, not just failures.
- The job ID, attempt number and token on **every** log line (log scopes).
- **Structured logging**: fields, not sentences, so logs can be queried.
- **Correlation IDs** and **distributed tracing** (spans) to follow one job across processes.

Appendix B: words to search

Term	One-line meaning
Read-modify-write race / TOCTOU	Checking, then acting, with a gap another process can use
Atomic conditional UPDATE	Check and change in one statement; the row count tells who won
SELECT FOR UPDATE	Lock the selected rows until my transaction ends
SKIP LOCKED / NOWAIT	Skip rows someone else has locked / error instead of waiting
Lease	Ownership that expires unless it is renewed
Heartbeat	The periodic renewal of a lease
Stale lock reclaim	Taking over a job whose lease has expired
Fencing token	A number incremented on every claim; old holders' writes are rejected
At-least-once execution	Every job runs one or more times, never zero
Idempotency / idempotency key	Running it twice has the same effect as running it once
Compare-and-set / conditional write	Write only if the value is still what I last read
Exponential backoff, jitter	Doubling delays between retries; randomness to spread them out
Thundering herd	Many clients retrying or waking at the same instant
Partial index	An index over only the rows matching a condition
Index / ORDER BY alignment	Making the sort order match the index so LIMIT can stop early
EXPLAIN / EXPLAIN ANALYZE	Show the query plan / run it and show real timings
Linked CancellationTokenSource	Cancels when any of its parent tokens cancels
Exception filter	catch ... when (condition)
Using declaration	Dispose at the end of the enclosing block
Append-only log / audit table	History stored as rows that are never updated
Natural key	An identifier that already exists in the data, e.g. (job ID, token)
IDOR / BOLA	Accessing another user's object by guessing its ID

Appendix C: the final exam

From PROJECTS.md, "You've got it when". Answer all three **cold**, without the code or these notes. Project 06 is done when I can.

1. Two workers run the claim query in the same millisecond. Why does neither get the other's rows?
2. Worker A claims job 5, hangs for 15 minutes, the lock is reclaimed, worker B runs job 5 and finishes. Worker A wakes up and writes its result. What stops it?
3. Why does the claim query's ORDER BY have to match the index?

Bonus questions an interviewer would ask next

1. Your fence works. Can a customer still receive two emails? Why?
2. A worker crashes mid-job. Walk me through exactly when, and by whom, the job gets re-run.
3. How do you know your claim query is still fast with ten million finished jobs in the table?
4. Your worker gets SIGTERM while a job is running. What happens to that job?
5. Why did you choose 10s / 30s / 3s for heartbeat, stale timeout and beat deadline?

Appendix D: primary docs to read

Topic	Where
Row locks, SKIP LOCKED, NOWAIT	PostgreSQL docs: SELECT, section "The Locking Clause" postgresql.org/docs/current/sql-select.html
Locks and isolation	PostgreSQL docs, chapter 13: 13.2 Transaction Isolation, 13.3 Explicit Locking
Indexes and ORDER BY	PostgreSQL docs, 11.4 Indexes and ORDER BY
Partial indexes	PostgreSQL docs, 11.8 Partial Indexes
Reading query plans	PostgreSQL docs, 14.1 Using EXPLAIN
Fencing tokens	Martin Kleppmann, "How to do distributed locking" (2016) martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html
Transactions in depth	Designing Data-Intensive Applications, chapter 7, "Transactions"
Idempotency keys	Stripe API docs, "Idempotent requests"
IDOR / BOLA	OWASP API Security Top 10 (2023), API1: Broken Object Level Authorization
Exception filters	Microsoft Learn: exception-handling statements, "Exception filters"
Using declarations	Microsoft Learn: "using statement"
Scope disposal	Microsoft Learn: Dependency injection guidelines, "Disposal of services"
Hosted services	Microsoft Learn: BackgroundService.ExecuteAsync remarks
Log scopes	Microsoft Learn: "Logging in C# and .NET", "Log scopes"